

中山大学专业课《程序设计I》

第二节 C中的数据

C Programming

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季

主要内容

- ❑ 一个简单的交互C程序
- ❑ 变量与基本变量类型
- ❑ 声明与赋值
- ❑ 标准输入输出
- ❑ 常量
- ❑ 储存类别

回顾

```
#include<stdio.h>
int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```

做点计算

```
#include<stdio.h>
int main(void)
{
    printf("%d\n", 2+5);
    return 0;
}
```

如何能在程序运行时输入数字，计算输入数字的计算结果？

一个简单的交互C程序

```
#include<stdio.h>

int main(void) {
    /*define vars and constants*/
    double radius, area;
    const double PI = 3.14;

    /*input radius*/
    printf("Input radius: ");
    scanf("%lf", &radius);

    /*compute area*/
    area = radius * radius * PI;

    /*output results*/
    printf("The area for the circle of radius %lf is %lf.\n",
        radius, area);

    return 0;
}
```

一个简单的交互C程序

```
/*compute area*/
#include<stdio.h>

int main(void) {
    /*define vars and constants*/
    double radius, area;
    const double PI = 3.14;

    /*input radius*/
    printf("Input radius: ");
    scanf("%lf", &radius);

    /*compute area*/
    area = radius * radius * PI;

    /*output results*/
    printf("The area for the circle of radius %lf is %lf.\n",
        radius, area);

    return 0;
}
```

这是一个简单的程序：

①声明变量和常量；

- radius和area称为“标识符”，用以识别不同变量
- 标识符只能由字母、数字和下划线组成，但数字不能出现在第一个位置上，C语言关键字也不可用于标识符
- double是“变量类型”，指明变量是浮点数（实数）
- PI=3.14是一个“赋值表达式”，“=”是赋值运算符，将“=”右边的值赋给左边的变量（与数学中“相等”含义不同！）
- const表示PI是一个常量（常量变量）

一个简单的交互C程序

```
/*compute area*/
#include<stdio.h>

int main(void) {
    /*define vars and constants*/
    double radius, area;
    const double PI = 3.14;

    /*input radius*/
    printf("Input radius: ");
    scanf("%lf", &radius);

    /*compute area*/
    area = radius * radius * PI;

    /*output results*/
    printf("The area for the circle of radius %lf is %lf.\n",
        radius, area);

    return 0;
}
```

这是一个简单的程序：

①声明变量和常量；

②从键盘接收一个实数作为半径；

- **%lf**表明输入为浮点数
- **&radius**告诉程序把输入的值赋给名为radius的变量
- **&**为取地址运算符，用以找到该变量的机制

一个简单的交互C程序

```
/*compute area*/
#include<stdio.h>

int main(void) {
    /*define vars and constants*/
    double radius, area;
    const double PI = 3.14;

    /*input radius*/
    printf("Input radius: ");
    scanf("%lf", &radius);

    /*compute area*/
    area = radius * radius * PI;

    /*output results*/
    printf("The area for the circle of radius %lf is %lf.\n",
        radius, area);

    return 0;
}
```

这是一个简单的程序：

- ①声明变量和常量；
- ②从键盘接收一个实数作为半径；
- ③**计算面积；**

一个简单的交互C程序

```
/*compute area*/
#include<stdio.h>

int main(void) {
    /*define vars and constants*/
    double radius, area;
    const double PI = 3.14;

    /*input radius*/
    printf("Input radius: ");
    scanf("%lf", &radius);

    /*compute area*/
    area = radius * radius * PI;

    /*output results*/
    printf("The area for the circle of radius %lf is %lf.\n",
        radius, area);

    return 0;
}
```

这是一个简单的程序：

- ①声明变量和常量；
- ②从键盘接收一个实数作为半径；
- ③计算面积；
- ④打印圆的面积。

一个简单的交互C程序

```
/*compute area*/
#include<stdio.h>

int main(void) {
    /*define vars and constants*/
    double radius, area;
    const double PI = 3.14;

    /*input radius*/
    printf("Input radius: ");
    scanf("%lf", &radius);

    /*compute area*/
    area = radius * radius * PI;

    /*output results*/
    printf("The area for the circle of radius %lf is %lf.\n",
        radius, area);

    return 0;
}
```

这是一个简单的程序：

- ①声明变量和常量；
- ②从键盘接收一个实数作为半径；
- ③计算面积；
- ④打印圆的面积。

变量 (Variables)

□ 在数学中，**变量**是用于指代特定值的**符号**

■ 例如

$$y = f(x) = 2x + 3$$

这里 y 与 x 都是变量，表示可变化的值

□ 在C语言中，**变量**是**储存**某个值或对象的**单元名称**

■ 例如

```
x = 3;  
y = 2 * x + 3;
```

这里 x 和 y 是两个存储单元。两个命令含义：第一步，数值3存入 x ；第二步， x 乘2加3后，结果存入 y

■ 变量的值在程序运行期间可以被改变

变量类型 (Types)

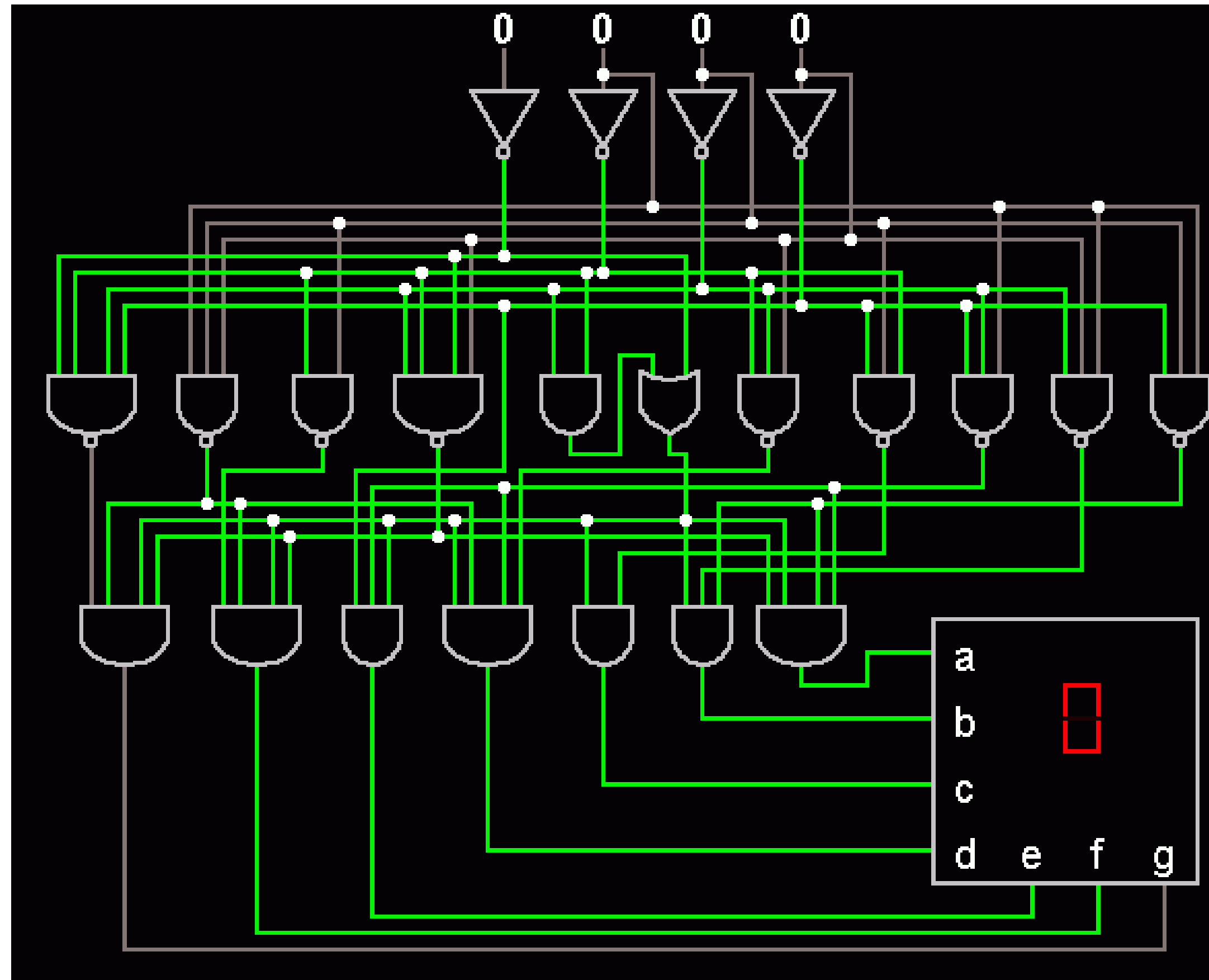
- C语言中变量只能储存数值，包括
 - **【整数类型】** 不含小数点和指数的数，如 1、-10、525
 - **【浮点类型】** 有小数部分的数，如 1.0、.14、1.2E4、5e-5、-3.e+4

整数类型

- ❑ 变量是储存空间，不同类型的变量所需储存空间的大小、方式不同，所以需要“**类型关键字**”声明
- ❑ 整数类型包括：**int unsigned short long char**
 - **【int】** 基本（有符号）整型
 - **【unsigned】** 无符号整型
 - **【short】** 短整型
 - **【long】** 长整型
 - **【char】** 字符型
- ❑ 同样合法：unsigned long, unsigned char, long long...

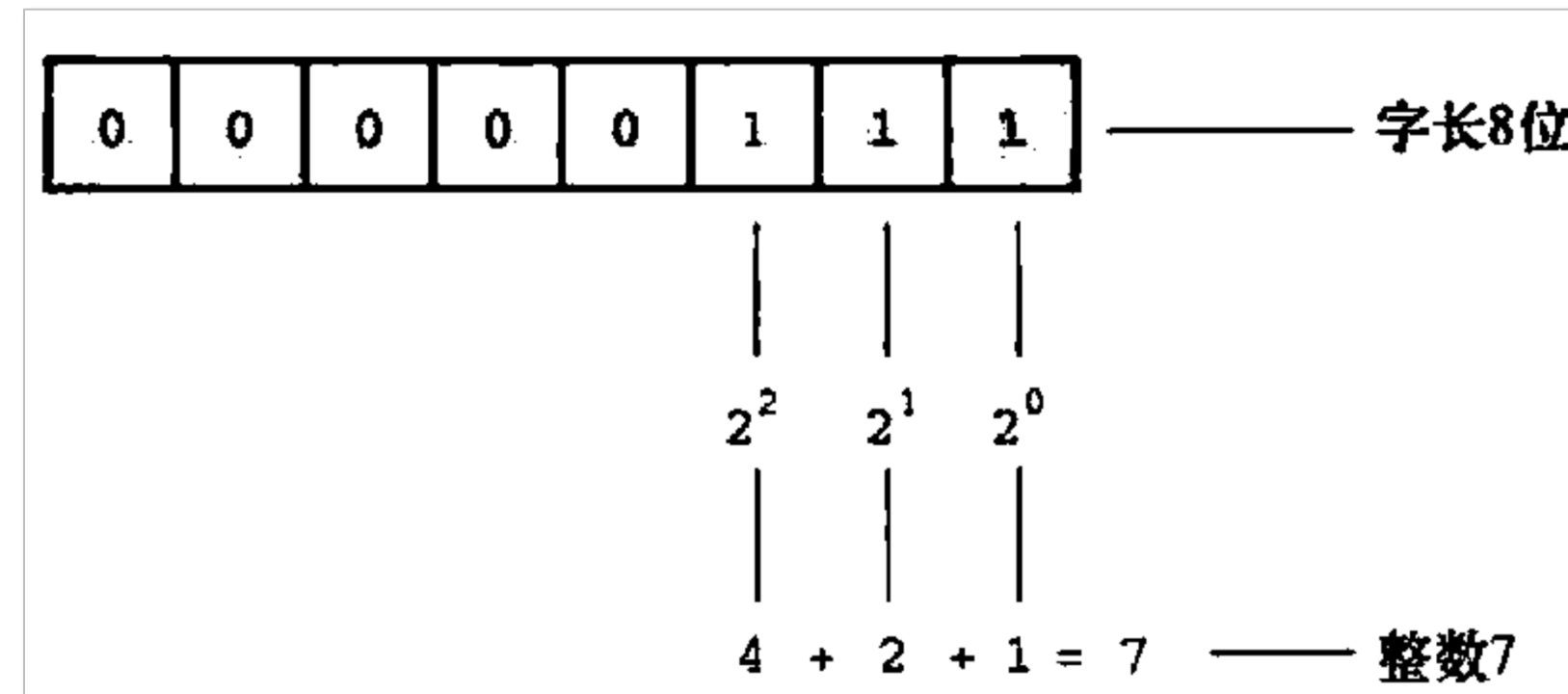
修饰关键字

计算机的数据表示（七段译码器）



整数类型

- 整数在计算机内部以二进制来存储



- 不同整数类型储存整型变量所用的**二进制位数**不同

整数类型

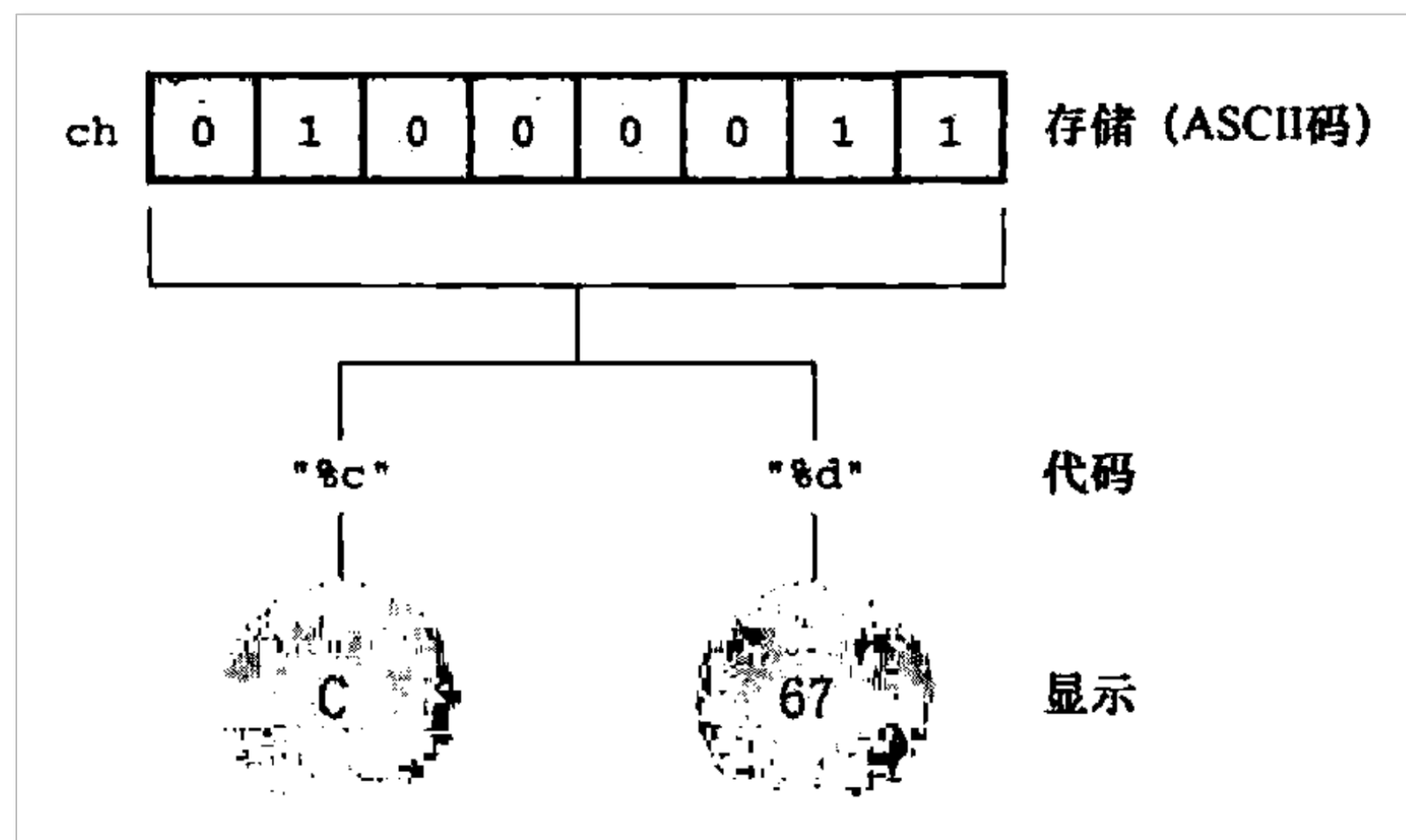
类型	类型标识符	字节	数值范围	数值范围
整型	[signed] int	4	-2147483648 ~ +2147483647	$-2^{31} \sim +2^{31}-1$
无符号整型	unsigned [int]	4	0 ~ 4294967295	$0 \sim +2^{32}-1$
短整型	short [int]	2	-32768 ~ +32767	$-2^{15} \sim +2^{15}-1$
无符号短整型	unsigned short [int]	2	0 ~ 65535	$0 \sim +2^{16}-1$
长整型	long [int]	4	-2147483648 ~ +2147483647	$-2^{31} \sim +2^{31}-1$
无符号长整型	unsigned long [int]	4	0 ~ 4294967295	$0 \sim +2^{32}-1$
长长整型	long long [int]	8	-9223372036854775808 ~ 9223372036854775807	$-2^{63} \sim +2^{63}-1$
无符号长长整型	unsigned long long [int]	8	0 ~ 18446744073709551616	$0 \sim +2^{64}-1$
字符型	[signed] char	1	-128 ~ +127	$-2^7 \sim +2^7-1$
无符号字符型	unsigned char	1	0 ~ +255	$0 \sim +2^8-1$

□ 【思考】运行以下代码会输出什么？

```
unsigned num = -1;
printf("num=%u\n", num);
```

特殊整型：char类型

- ❑ char用于储存字符，如：A、B、1、\n
- ❑ 实质上char类型存储的是整数类型，占1字节（8比特）



- ❑ char类型的声明与初始化：char ch = 'A';
 - 这里'A'为字符常量，实为int类型（占4字节）

浮点类型

□ 浮点类型包括： **float double**

- **【float】** 单精度型，一般为4字节，至少6位有效数字
- **【double】** 双精度型，一般为8字节，至少10位有效数字

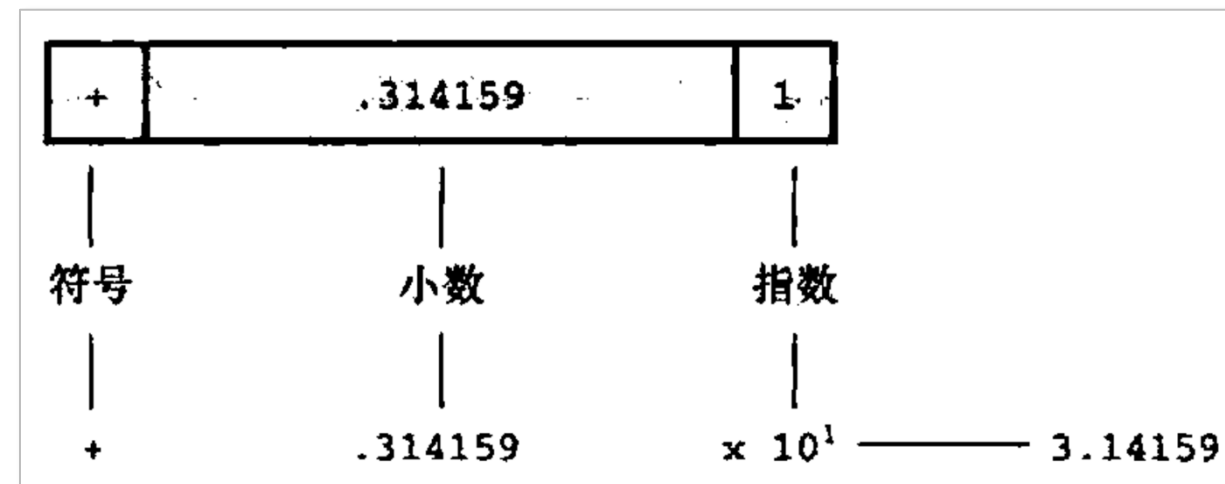
□ 考虑以下程序：

```
float a, b;  
b = 2e20 + 1.0;  
a = b - 2e20;  
printf("a = %f\n", a);
```

□ 任意不同实数之间都有无穷多个数， **浮点数通常只是实际值的近似值**

浮点类型

□ 浮点类型的储存方式



□ 编译器假定浮点型常量是double

```
float x = 0.123456789;  
printf("x = %.10f\n", x);  
printf("%.10f\n", 0.123456789);
```

□ 在浮点常量后添加后缀f/F或L改变默认设置，如：0.1F

常见基本变量类型

类型	类型标识符	字节	数值范围	数值范围
整型	[signed] int	4	-2147483648 ~ +2147483647	$-2^{31} \sim +2^{31}-1$
无符号整型	unsigned [int]	4 2 []表示可省略	0 ~ 4294967295	$0 \sim +2^{32}-1$
短整型	short [int]		-32768 ~ +32767	$-2^{15} \sim +2^{15}-1$
无符号短整型	unsigned short [int]	2	0 ~ 65535	$0 \sim +2^{16}-1$
长整型	long [int]	4	-2147483648 ~ +2147483647	$-2^{31} \sim +2^{31}-1$
无符号长整型	unsigned long [int]	4	0 ~ 4294967295	$0 \sim +2^{32}-1$
长长整型	long long [int]	8	-9223372036854775808 ~ 9223372036854775807	$-2^{63} \sim +2^{63}-1$
无符号长长整型	unsigned long long [int]	8	0 ~ 18446744073709551616	$0 \sim +2^{64}-1$
字符型	[signed] char	1	-128 ~ +127	$-2^7 \sim +2^7-1$
无符号字符型	unsigned char	1	0 ~ +255	$0 \sim +2^8-1$
单精度型	float	4	$-3.4 \times 10^{38} \sim 3.4 \times 10^{38}$	$-3.4 \times 10^{38} \sim 3.4 \times 10^{38}$
双精度型	double	8	$-1.7 \times 10^{308} \sim 1.7 \times 10^{308}$	$-1.7 \times 10^{308} \sim 1.7 \times 10^{308}$
长双精度型	long double	8	$-1.7 \times 10^{308} \sim 1.7 \times 10^{308}$	$-1.7 \times 10^{308} \sim 1.7 \times 10^{308}$

如何知道类型大小? sizeof()

变量的声明

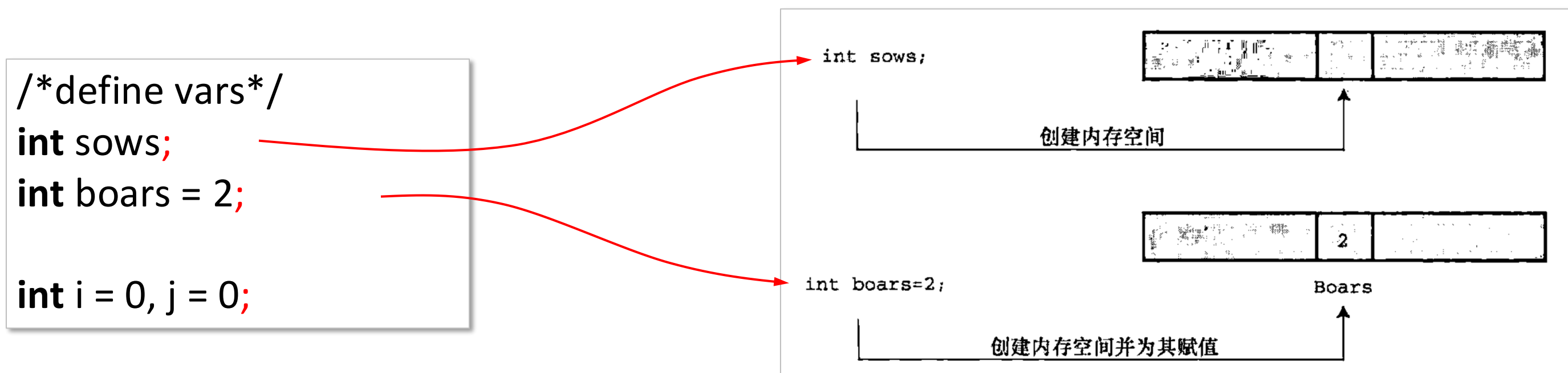
- ❑ 变量声明就是向程序表明变量的**类型和名称**，并**创建、标记存储空间**
- ❑ 具体语法：**<类型> <变量名>;**
 - <类型> 代表有效的C数据类型，<变量名> 可以由一个或多个标识符名称组成，多个标识符之间用 “,” 分隔
- ❑ 几个有效的声明：

```
/*define vars and constants*/  
double radius, area;  
  
int i, j;  
char c;
```
- ❑ 通常，我们**使用熟悉的单词作为变量名称**，例如radius、height等

变量初始化

□ 变量可以在声明时初始化（指定初值）

■ 初始化器由一个赋值运算符，后跟一个常量表达式组成，例如：



□ 注意，这样对于a、b的初始化是不允许的！

~~int a = b = 3;~~

变量使用与赋值

❑ 定义或声明后的变量，可用“=”赋值，称为赋值运算符

❑ 定义的变量可以在表达式中使用，如

```
/*define vars and constants*/  
double radius = 5;  
double area;  
  
area = radius * radius * 3.14;
```

初始化!

赋值!

最后一个语句是赋值语句（表达式）：它使用radius变量的值计算出圆面积，通过赋值把结果值写入左边area变量中

❑ 注意：当把一个类型数值初始化给不同类型变量时，会发生隐式类型转换，如

```
int num = 1.99;  
float pi = 3.1415926536;
```


输入 (Input) 与输出 (Output)

- 如何让我们看到变量中的内容?
- **输出**: 把内存中变量或表达式计算结果在显示器、打印机或磁盘文件等外部设备读出
 - 默认的输出设备称为**标准输出** (Standard Output) , 一般是**控制台**
 - 操作函数: `printf`, `putchar`, `puts`
- **输入**: 从外部设备读入数据到内存变量
 - 默认的输入设备称为**标准输入** (Standard Input) , 一般是**键盘**
 - 操作函数: `scanf`, `getchar`, `gets`
- 这些基本的操作函数在 `stdio.h` 中定义

变量输出： printf()

```
#include<stdio.h>
```

```
int main(void) {
```

```
    double radius = 5.0;
```

```
    double area;
```

```
    area = radius * radius * 3.14;
```

```
    printf("The area for the circle of radius %lf is %lf.\n",  
          radius, area);
```

```
    return 0;
```

```
}
```

使用 printf 输出到标准输出设备

语法格式：

- `printf(格式化字符串,变量/常量/表达式项)`
- 格式化字符串中**转换说明符**:
 - `%d` 与 int 对应
 - `%ld` 与 long 对应
 - `%f %lf` 与 float, double 对应
 - `%c` 与 char 对应（输出字符）

注意：

- ① 转换说明符指明了一行中打印的位置，与变量列表中变量一一对应；
- ② C99起，`%f`和`%lf`是完全相同的；
- ③ `printf`当缓冲区满、遇换行符或者需要用户输入时，才会把缓冲区内容打印到屏幕上。

变量输出： printf()

□ 注意

- ① 转换说明符的个数与参数个数一致；
- ② 转换说明符的类型与参数类型一致。

□ 如果个数不匹配会发生什么？为什么？

```
#include<stdio.h>

int main(void) {

    double radius = 5.0;
    double area;

    area = radius * radius * 3.14;

    printf("Area is %lf.\nRadius is %lf.\n", radius);

    return 0;
}
```

转换不匹配

□ 参数类型不匹配会发生什么？为什么？

```
#include<stdio.h>

int main(void) {
    /*define vars and constants*/
    double radius = 5.0;
    double area;

    /*compute area*/
    area = radius * radius * 3.14;

    /*output results*/
    printf("The area for the circle of radius %d is %lf.\n",
           radius, area);

    return 0;
}
```

□ 转换实际上是翻译，%d是把给定的值翻译成十进制整数并打印

格式化 (format) 与转换说明符

- 格式字符串由普通字符（除了%）和转换说明符构成

```
/*format testing*/
#include<stdio.h>

int main(void) {
    double radius = 5;

    printf("%lf\n", radius);
    printf("%10lf\n", radius);
    printf("%10.2lf\n", radius);
    printf("%-10.2lf\n", radius);
    printf("%+10.2lf\n", radius);
    printf("% 10.2lf\n", radius);
    printf("%010.2lf\n", radius);

    return 0;
}
```

课堂练习

□ 将下面自然语言翻译成C代码：

1. 第一步：使用一个名为 miles 初始值为 100 的实数变量
2. 第二步：将 miles 乘 1.609 并将它赋值给一个名为 kilometers 的变量
3. 第三步：显示 kilometers 的值，小数部分保留两位

变量输入：scanf()

```
/*io simple*/
#include<stdio.h>

int main(void) {
    int a,b,c;

    printf("请输入: ");
    scanf("%d%d,%d",&a,&b,&c);
    printf("%d,%d,%d\n",a,b,c);
    return 0;
}
```

使用 scanf 从标准输入读入

语法格式:

- printf(格式化字符串,变量地址列表)
- 格式化字符串中转换说明符:
 - %d 与 int 对应
 - %ld 与 long 对应
 - %f 与 float 对应
 - %lf 与 double 对应
 - %c 与 char 对应 (输出字符)

注意:

- ① 格式化字符中普通字符必须与输入的一样;
- ② 变量数量和类型必须与格式化字符串需要的类型一致;
- ③ 必须是变量的地址。

思考

□ scanf()根据不同转换说明符 (%d %f %s %c) 读取相同内容会发生什么?

" -13.45e12# 0"

printf 和 scanf 函数格式简表

- ❑ printf调用格式: printf ("<格式化字符串>", <参量表>);
- ❑ scanf调用格式: scanf ("<格式化字符串>", <地址表>);
- ❑ 常见格式化控制符

%c	读单字符
%d	读十进制整数
%l	读十进制Long型整数
%ll	读十进制Long Long型整数
%i	读十进制、八进制、十六进制整数
%e	读浮点数
%E	读浮点数
%f	读浮点数

%g	读浮点数
%G	读浮点数
%o	读八进制数
%s	读字符串
%x	读十六进制数 (其中abcdef小写, 大写时会被忽略)
%X	读十六进制数 (其中ABCDEF大写, 小写时会被忽略)
%p	读指针值

一个简单、完整的C程序

```
/*compute area*/
#include<stdio.h>

int main() {
    /*define vars and constants*/
    double radius, area;
    const double PI = 3.14;

    /*input radius*/
    printf("input radius:");
    scanf("%lf",&radius);

    /*compute area*/
    area = radius * radius * PI;

    /*output results*/
    printf("the area for the circle of radius %lf is %lf",
           radius, area);

    return 0;
}
```

对于单一main函数的程序，多数会呈现四个步骤：

1. 定义变量
2. 输入数据
3. 处理或计算
4. 输出数据

常量 (Constant)

- 常量是固定值，在程序执行期间不会改变
- 常量主要包括整数常量、浮点常量、字符常量、字符串常量与符号常量

整数常量

- ❑ 整数常量可以是十进制，也可以是二进制、八进制或十六进制
- ❑ 用整数前缀表示非十进制，例如：
 - 二进制：前缀0b (C23)
 - 八进制：前缀0
 - 十六进制：前缀0x
- ❑ 用整数后缀表示无符号整数 (u/U) 以及长整数 (l/L)

212	/* 合法的 */
215u	/* 合法的 */
0xFeeL	/* 合法的 */
078	/* 非法的: 8 不是八进制的数字 */
032UU	/* 非法的: 不能重复后缀 */

```
int x = 100;
printf("dec = %d, octal = %o, hex = %x\n", x, x, x);
printf("dec = %d, octal = %#o, hex = %#x\n", x, x, x);
```

详细说明参见: [整数常量 - cppreference.com](http://cplusplus.com/cppreference.com)

浮点常量

- 浮点常量由**整数部分**、**小数点**、**小数**和**指数**部分组成，可以使用小数形式或者指数形式来表示浮点常量
- 小数形式表示时，必须包含整数部分、小数部分，或同时包含两者
- 指数形式表示时，E/e前面为尾数部分，必须有数，e后为指数部分，必须为整数

```
3.14159      /* 合法的 */
314159E-5L   /* 合法的 */
510E         /* 非法的: 不完整的指数 */
210f         /* 非法的: 没有小数或指数 */
.e55         /* 非法的: 缺少整数或分数 */
```

字符串常量

- 含义：连续多个字符组成的字符序列
- 表示：括在双引号中，可以是图形字符，也可以是转义符

```
"abc 123*#"
"\x61 \x62\x63\061 \62\063\x2a\043"
"\r\n\t\\A\\t\x1b\\"1234\xft \x2f\33"
```

- 思考：以下语句会输出什么？

```
printf("\r\n\t\\A\\t\x1b\\"1234\xft \x2f\33\n");
```

符号常量

□ C允许用符号定义常量

```
#define TAXRATE 0.015
```

```
#define PI 3.14159265
```

```
#define TAB '\t'
```

- 编译时替换：程序编译时预处理器会将 TAXRATE 替换为 0.015

□ C90开始允许使用 `const` 关键字限定变量为只读

```
const int MONTHS = 12;
```


字符串 (String) 的定义、输入与输出

□ 示例

```
#include<stdio.h>

int main(void)
{
    char str[233];
    printf("请输入：");
    scanf("%s", str);
    printf("%s\n", str);

    return 0;
}
```

- 读取字符串的时候**不需要加&**，因为这里str本身就是**指针**，已经可以认为是地址了

字符串逆序输出

7-5 输入字符串逆序输出

分数 5

[查看题目详情](#) [全屏浏览](#) [切换布局](#)

输入一个字符串，逆序将此字符串按字符一个一个输出

输入格式:

输入字符串，不超过100个字符。

输出格式:

逆序按一个一个字符输出，中间用-隔开。

```
#include <stdio.h>
#include <string.h>

int main(void){
    char str[101];
    scanf("%s", str);

    int len = strlen(str);
    printf("%c", str[len-1]);

    for (int i=len-2; i>=0; i--)
        printf("-%c", str[i]);

    printf("\n");
    return 0;
}
```

存储类别

- ❑ 存储类别定义了C程序中变量/函数的**可访问性**和**生命周期**，这些说明符放置在它们所修饰的类型之前
- ❑ 下面是C中主要用的存储类别
 - 自动变量
 - 寄存器变量
 - 静态变量
 - 外部变量

自动变量

- ❑ 说明符: **auto**
- ❑ auto限定变量只能在函数/块内部使用
- ❑ 自动变量是所有局部变量（函数内部定义的变量）**默认**的存储类，下面两个声明等价：

```
{  
    int x;  
    auto int y;  
}
```


寄存器变量

- ❑ 说明符: **register**
- ❑ register用于“定义”存储在**寄存器**中而不是内存中的局部变量

```
{  
    register int x;  
}
```

- ❑ 意味着变量的最大尺寸等于寄存器的大小，且不能对它应用一元的 **'&'** 运算符（因为它没有内存位置），所以下面这样写是错的

```
{  
    register int x;  
    int *a=&x;  
}
```

静态变量

- ❑ 说明符：**static**
- ❑ 用static修饰变量后，变量的生命周期变成程序结束为止
- ❑ 用static修饰全局变量，则全局变量的**作用域**从原来的整个程序，变为静态全局变量所在的**文件内部**；修饰局部变量，作用域不变
- ❑ 用static修饰的变量，存储位置是**全局数据区**（即静态区）
- ❑ 注：修饰之前，局部变量存储在**栈区**

外部变量（全局变量）

- 外部变量的声明在所有函数之外，对所有的程序文件都是可见的
- `extern`说明符提供了一个全局变量的“引用”，当有多个文件且定义了外部变量（或函数）时，可以在其他文件中使用`extern`来得到已定义的变量（或函数）

第一个文件

```
C a.c > ...  
1  int a=1;  
2  
3
```

第二个文件

```
C b.c > ...  
1  #include <stdio.h>  
2  
3  extern int a;  
4  ✓ int main()  
5  {  
6  |   printf("a=%d",a);  
7  | }
```

extern存储类

- ❑ 如果把a.c中a的定义加上static就会编译错误，因为用static修饰后对于其他文件就是不可见的了

第一个文件

```
C a.c > ...  
1 static int a=1;  
2  
3
```

编译结果

```
PS C:\Users\86147\Desktop> gcc b.c a.c -o Moon.exe  
C:\Users\86147\AppData\Local\Temp\ccAHgCvX.o:b.c:(.rdata$.refptr.a[.refptr.a]+0x0): undefined reference to `a'  
collect2.exe: error: ld returned 1 exit status
```


类型转换

```
printf("%f\n", 10);

int i = 10;
double d = i;
printf("d = %f\n", d);
```

- 当在表达式中使用混合类型时，C语言会进行自动类型转换
 - 两种类型运算时，两个值会转换为两种类型的更高级别
 - 在赋值表达式中，计算结果会转换为被赋值变量类型

```
double d = 10;
printf("d = %f\n", d);

int i = 3.14;
printf("i = %d\n", i);
```

强制类型转换

- ❑ 强制类型转换是把变量从一种类型转换为另一种数据类型
- ❑ 例如: `(int)x` 把一个 `long` 类型的值 `x` 转到一个 `int` 中

```
int a = 21, b = 4;  
double c,d;  
  
c = (double) a / b;  
d = (double)(a / b);  
  
printf("c = %f\n", c);  
printf("d = %f\n", d);
```

- ❑ 这里输出结果会有差异，是因为对于整数`a`、`b`默认是整数的除法，需要强制转换后才能变成实数的除法

课堂练习： 计算圆柱体体积

编写一个读取圆柱的半径和高并，利用下面的公式计算圆柱体底面积和体积的程序：

$$\begin{aligned} area &= radius \times radius \times \pi \\ volume &= area \times length \end{aligned}$$

运行示例：

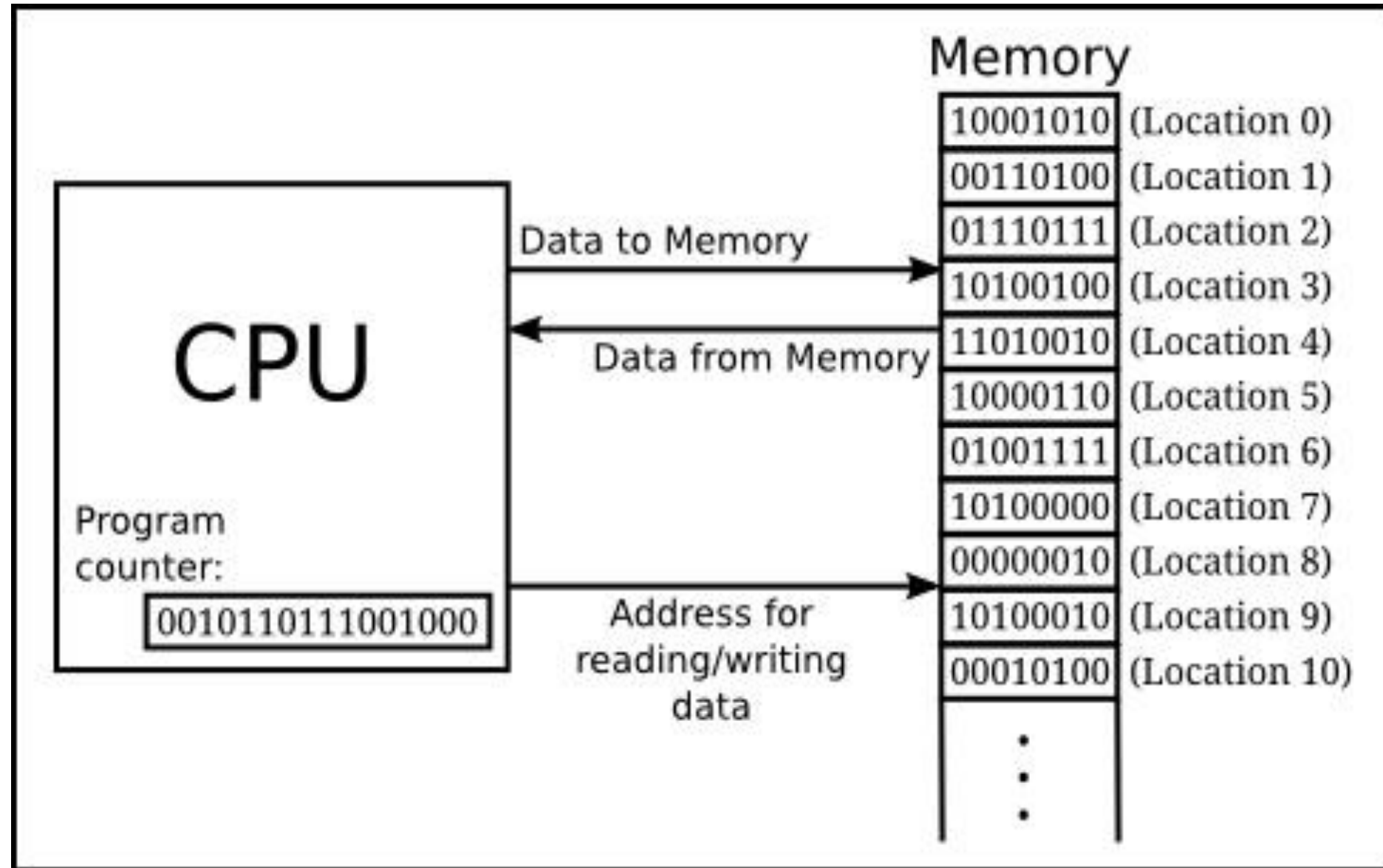
```
Enter the radius and length of a cylinder: 5.5,12
The area is 95.03
The volume is 1140.40
```




中山大學
SUN YAT-SEN UNIVERSITY

补充1：整数及其表示

回顾



`double radius = 5;`

- 在计算机内部:
 - 5在内存中如何用0、1来表示?
 - radius 与 &radius 含义?
 - double 和 int 类型的 5, 在内存中一样吗?

整数与变量

```
/*unsigned int add*/
#include <stdio.h>

int main() {
    // Add two int
    unsigned int i,j;
    i = 2;
    j = 3;
    // 详细说明见 : https://zh.cppreference.com/w/c/io/fprintf
    printf("%u add %u equal %u\n",i,j,i+j);
    return 0;
}
```

- `unsigned int` 声明 (declaration) `i,j` 两个整数变量
- 类似两个房间，标牌是 `i` 和 `j`
- `i`和`j`又称为变量名，也是保存值的代称

= 是赋值运算符，给 `j` “房间” 储存数值3

- “`%u add %u equal %u\n`” 是格式化模板
- `%u` 中 `%` 是引导符，`u` 代表无符号整型
- 这个模板要填3个数字：`i,j,i+j`

若将 `j=3;` 修改为 `j=-3;` 输出结果是？

2 add 4294967293 equal 4294967295

数 (Number)

□ 自然数 (Natural Numbers) , 无符号整数 (Unsigned Integers)

- 零以及通过不断在零上加一而得到的任意数
- 例如: 100, 0, 45645, 32

□ 负数 (Negative Numbers)

- 小于 0 的数, 带有负号 “-”
- 例如: -24, -1, -45645, -32

□ 整数 (Integers)

- 自然数、负数、零
- 例如: 249, 0, - 45645, - 32

□ 有理数 (Rational Numbers)

- 整数, 或两个整数的商
- 例如: -249, -1, 0, $3/7$, $-2/5$

ASCII码

高四位 低四位		ASCII非打印控制字符										ASCII 打印字符													
		0000					0001					0010		0011		0100		0101		0110		0111			
		0					1					2		3		4		5		6		7			
		+进制	字符	ctrl	代码	字符解释	+进制	字符	ctrl	代码	字符解释	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符
0000	0	0	BLANK NULL	^@	NUL	空	16	▶	^P	DLE	数据链路转意	32		48	0	64	@	80	P	96	`	112	p		
0001	1	1	☺	^A	SOH	头标开始	17	◀	^Q	DC1	设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q		
0010	2	2	☹	^B	STX	正文开始	18	↕	^R	DC2	设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r		
0011	3	3	♥	^C	ETX	正文结束	19	!!	^S	DC3	设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s		
0100	4	4	♦	^D	EOT	传输结束	20	¶	^T	DC4	设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t		
0101	5	5	♣	^E	ENQ	查询	21	♠	^U	NAK	反确认	37	%	53	5	69	E	85	U	101	e	117	u		
0110	6	6	♠	^F	ACK	确认	22	■	^V	SYN	同步空闲	38	&	54	6	70	F	86	V	102	f	118	v		
0111	7	7	●	^G	BEL	震铃	23	↕	^W	ETB	传输块结束	39	'	55	7	71	G	87	w	103	g	119	w		
1000	8	8	◼	^H	BS	退格	24	↑	^X	CAN	取消	40	(	56	8	72	H	88	X	104	h	120	x		
1001	9	9	○	^I	TAB	水平制表符	25	↓	^Y	EM	媒体结束	41	)	57	9	73	I	89	Y	105	i	121	y		
1010	A	10	◻	^J	LF	换行/新行	26	→	^Z	SUB	替换	42	*	58	:	74	J	90	Z	106	j	122	z		
1011	B	11	♂	^K	VT	竖直制表符	27	←	^[	ESC	转意	43	+	59	;	75	K	91	[	107	k	123	{		
1100	C	12	♀	^L	FF	换页/新页	28	└	^\ ^_	FS	文件分隔符	44	,	60	<	76	L	92	\	108	l	124			
1101	D	13	♪	^M	CR	回车	29	↔	^] ^_	GS	组分隔符	45	-	61	=	77	M	93	]	109	m	125	}		
1110	E	14	🎵	^N	SO	移出	30	▲	^6	RS	记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~		
1111	F	15	☼	^O	SI	移入	31	▼	^-	US	单元分隔符	47	/	63	?	79	O	95	_	111	o	127	␣ ^Back space		

CSDN @ ZhangChen

自然数

□ “642” 表示多少个1?

■ $600 + 40 + 2$?

■ 也可能: $384 + 32 + 2$?

■ 也可能是: $1536 + 64 + 2$?

进位制/位值计数法 (Positional Notation)

- 为什么?
- 642 用基 (Base) 为 10 位值计数法:

$$6 \times 10^2 + 4 \times 10^1 + 2 \times 10^0 = (642)_{10}$$

这个数表示基为10

指数表示在数值中的
位置

进位制/位值计数法 (Positional Notation)

□ 642的十进制表示:

$$6 \times 10^2 + 4 \times 10^1 + 2 \times 10^0 = (642)_{10}$$

□ R进制的公式表示:

$$d_n \times R^{n-1} + d_{n-1} \times R^{n-2} + \cdots + d_2 \times R + d_1 = (d_n d_{n-1} \cdots d_1)$$

where $d_i \in [n]$

再看这个例子

□ 用“642”表示多少个1？

■ $600 + 40 + 2$?

■ 也可能: $384 + 32 + 2$?

■ 也可能是: $1536 + 64 + 2$?

□ 分别是多少进制？

■ $6 \times 8^2 + 4 \times 8^1 + 2 = (418)_{10}$

■ $6 \times 16^2 + 4 \times 16^1 + 2 = (1602)_{10}$

■ $6 \times 13^2 + 4 \times 13^1 + 2 = (1068)_{10}$

■ ...

进位制/位值计数法 (Positional Notation)

□ **十进制 (Decimal)** 的基 (Base) 是10, 有10个数字:

■ 0,1,2,3,4,5,6,7,8,9

□ **二进制 (Binary)** 的基是2, 有2个数字:

■ 0,1

□ **十六进制 (Hexadecimal)** 的基是16, 有16个数字:

■ 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

□ **八进制 (Octal)** 的基是?, 有? 个数字:

■ ?

练习

□ 计算以下数的十进制值

$(143)_8$

□ 答案：99

十进制转R进制

除R取余数倒排法

例如： $(89)_{10} = (1011001)_2$

2	89	取余	
2	44	 1	最低位
2	22	 0	
2	11	 0	
2	5	 1	
2	2	 1	
2	1	 0	
	0	 1	最高位

例如： $(219)_{10} = (DB)_{16}$

16	219	取余
16	13	 11 → B
	0	 13 → D

十进制转十六、八进制

□ 十六进制数 **DEF** 等于？ 十进制数

```
/*int-hex-oct*/  
#include <stdio.h>  
  
int main(void) {  
    unsigned int i = 3567;  
    printf("Hex %#x, Oct %#o\n",i,i);  
    return 0;  
}
```

□ 我们能用 “%b” 直接打印二进制吗？

二进制与十六进制、八进制的关系

Binary	Octal	Hexa
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7

Binary	Octal	Hexa
1000		8
1001		9
1010		A
1011		B
1100		C
1101		D
1110		E
1111		F

$$\begin{aligned}(\text{DEF})_{16} &= (1101\ 1110\ 1111)_2 \\ &= (110\ 111\ 101\ 111)_2 \\ &= (6757)_8\end{aligned}$$

发现什么规律?

练习

- 计算以下数的二进制、八进制、十六进制值

$(143)_{10}$

- 二进制：10001111

- 八进制：217

- 十六进制：8F

将十六进制与八进制数赋值给变量

```
/*hex-oct-int*/
#include <stdio.h>

int main() {
    unsigned int i = 0xDEF;
    unsigned int j = 06757;
    printf("i in Decimal is %u\n",i);
    printf("j in Decimal is %u\n",j);
    return 0;
}
```

0x 开头表示是 十六进制 的数值

0 开头表示是 八进制 的数值

变量i能保存什么范围的数?

字节和无符号数据类型

□ 字节 (Byte)

- 8位二进制数表示范围 $0 \dots 2^8-1$

□ 字 (Word)

- CPU一次处理的二进制位数
- 对应CPU类型：16、32、64位

□ 无符号整数 (unsigned int)

- 4字节，范围 $0 \dots 2^{32}-1$

```
#include <stdio.h>
#define UINT_MAX -1
typedef unsigned char byte;

int main(void)
{
    byte b = UINT_MAX;
    unsigned int i = UINT_MAX;
    unsigned long l = UINT_MAX;
    printf("Maxmum byte is %hhx , %hhu\n",b,b);
    printf("Maxmum int is %x , %u\n",i,i);
    printf("Maxmum long is %lx , %lu\n",l,l);

    return 0;
}
```

为什么可以用-1表示不同类型无符号整数的最大数值？

负数 (Negative) 的表示

❑ 二进制的补码 (Complement)

■ $\text{Negative}(b) = 2^k - b$ 其中 k 是 b 的位数

❑ 负数用对应正数的补码来表示

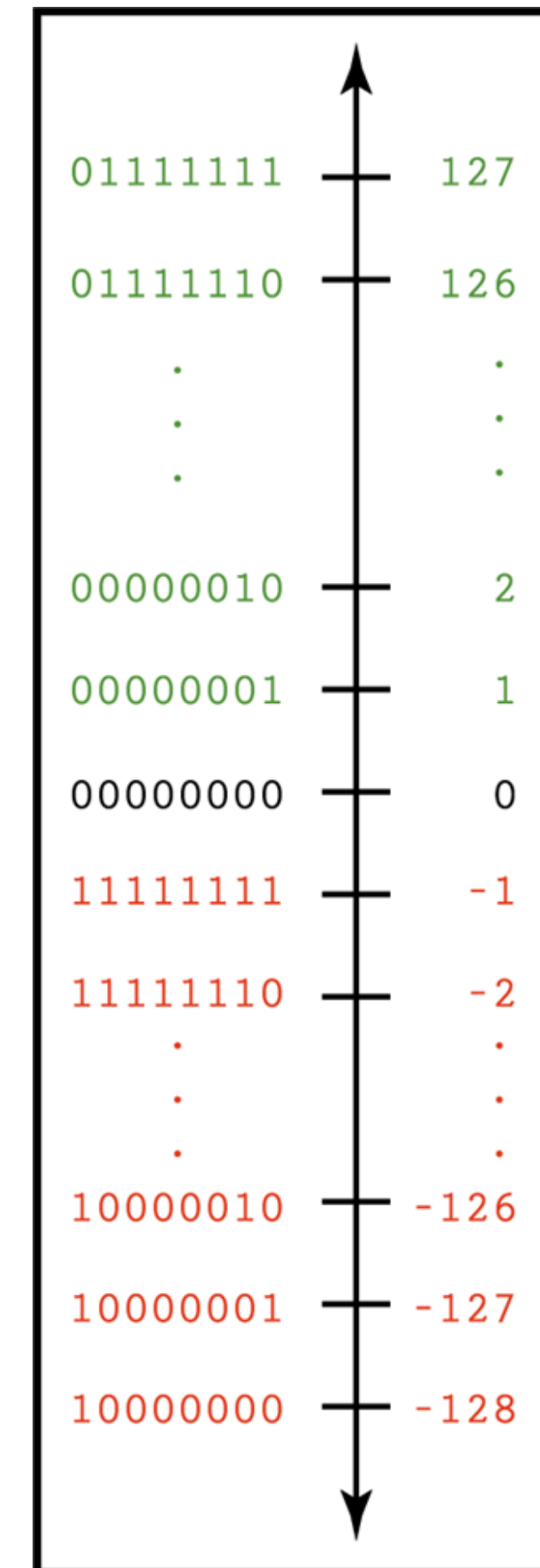
❑ 例如：对于 1 byte 的数

■ 正数 + 对应的负数 = 256

■ 负数第一位是1正数是0，所以它称为符号位

■ -1即是从第k位借位减1，一定是全111...111

❑ 22 = 00010110 的二进制负数表示是？



二进制补码的计算方法

□ 二进制的补码可以表示为

■ $\text{Negtive}(b) = (2^k - 1) - b + 1$

□ 00010110 的二进制负数表示

1111, 1111	-- $2^8 - 1$
- 0001, 0110	-- b求二进制
1110, 1001	-- 按位求反
+ 1	-- 加一
1110, 1010	-- 补码 -b

□ 二进制数的补码等于**按位求反再加一**

■ 因此在CPU中，可以用加法电路实现减法： $a - b = a + (-b)$

整数类型的特征

□ 字符型 (char)

- 1 个字节
- 表示范围 -128 ... 127

□ 短整型 (short)

- 2 个字节
- 表示范围 -32768 ... 32767

□ 整型 (int) , 长整型 (long)

- 4 个字节
- 表示范围 $-2^{31} \dots 2^{31}-1$

```
/*short-max-min*/
#include<stdio.h>
#define UINT_MAX -1
#define SHORT_MAX (unsigned short) UINT_MAX >> 1
#define SHORT_MIN ~SHORT_MAX

int main(void) {
    short i,j;
    i = 0x8000; // SHORT_MIN;
    j = 0x7FFF; // SHORT_MAX;
    printf("Minmum is %hX , %d\n",i,i);
    printf("Maxmum is %hX , %d\n",j,j);
    return 0;
}
```

参数的大小的“长度”修饰符, h用于short, l用于long

转换格式说明符, d用于整数, u用于无符号整数

练习

1. 计算 -7 的char类型的十六进制值
2. 分别写出以下各变量的八进制值:
 - char x=3;
 - char y=-6;
 - char z=x-y;

□ 答案:

1. F9
2. 3, 372, 11

溢出 (Overflow)

```
/*overflow*/
#include <stdio.h>

int main(void) {
    char k = 127; //0x7F;
    char m = k + 10;
    char n = k + k + 10;
    printf("%d, %d, %d\n", k+10, m, n);
}
```

表达式是运算符及其操作数的序列，它指定一个运算

<https://zh.cppreference.com/w/c/language/expressions>

这些表达式的结果是整数，当赋值给一个位数少的变量时，直接**抛弃高位**。例如：

- $k+10 = 0x89$ ，作为32位整数为137，作为8位整数为-119
- $k+k+10 = 0x108$ ，赋值n时，仅保留0x08

输出是：

137, -119, 8

溢出 (Overflow)

□ 另一个例子

```
/*overflow constant*/  
#include <stdio.h>  
  
int main(void) {  
    char k = 255+1;  
    printf("%x", k);  
}
```

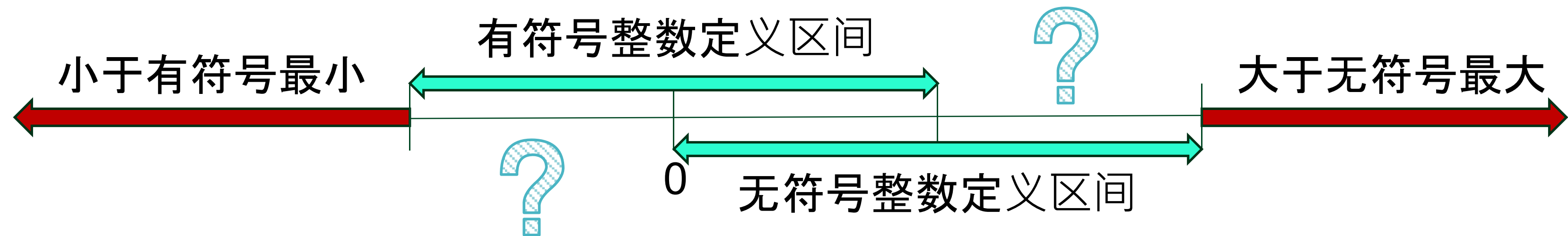

溢出 (Overflow) 的定义与检出

❑ 整数溢出

- 赋予变量的值超出变量类型定义的范围

❑ C语言语法层面可对**常数赋值**进行溢出检查

- 例如 `char i = 129;`



❑ 表达式赋值则按转换规则，常不做做任何检查

❑ C语言开发者有义务保证不发生溢出！

历史上最著名的缺陷（BUG）与溢出

- ❑ 1996年6月4日，对于Ariane 5火箭的初次航行来说，这样一个错误产生了灾难性的后果——发射后仅仅37秒，火箭偏离它的飞行路径解体并爆炸，6亿美元付之一炬

❑ 错误分析

- During execution of a **data conversion** from **64-bit floating point** to **16-bit signed integer value**. The floating point number which was converted had a value greater than what could be represented by a 16-bit signed integer. This resulted in an Operand Error.



参考阅读：[C语言的整型溢出问题](#) | 酷壳 - CoolShell

整型常量 (Constants) 的书写形式

- 允许整数类型的值直接用于表达式

- 语法

- 十进制常量 整数后缀 (可选)
- 八进制常量 整数后缀 (可选)
- 十六进制常量 整数后缀 (可选)

- 整数后缀

- 无后缀
- u/U: 无符号数
- l/L: 长整型
- Lu/LU: 无符号长整型
- Ll/LL: 长长整型

```
int d = 42;  
int o = 052;  
int x = 0x2a;  
int X = 0X2A;  
long l = 420L;  
long long ll = 052LU;
```

详细参考:
[整数常量 - cppreference.com](http://cppreference.com)



中山大學
SUN YAT-SEN UNIVERSITY

补充2：浮点数及其表示

浮点数 (Floating Number) 与格式化

```
/*floating format*/  
#include <stdio.h>  
  
int main() {  
    double x,y;  
    x = -2.263;  
    y = 13.367;  
    double z = x + y;  
    // 详细说明见: https://zh.cppreference.com/w/c/io/fprintf  
    printf("%-8.2f add %08.2f equal %.4e\n",x,y,z);  
    return 0;  
}
```

double 表示浮点类型，这里定义了变量 x 和 y

变量赋值

精度4位，科学计算法输出

用0补齐，宽度8，精度2位，浮点数输出

左对齐，宽度8，精度2位，浮点数输出

输出是：

-2.26 add 00013.37 equal 1.1104e+001

浮点数数据类型与常量表示

□ 双精度数 (double)

- 8个字节 (64位)
- 有效数字 15
- 指数部分 $10^{-307} \sim 10^{308}$

```
double d = 42.0;  
double o = 4.2e1;  
float x = 42.0f;  
float y = 4.2e1f;
```

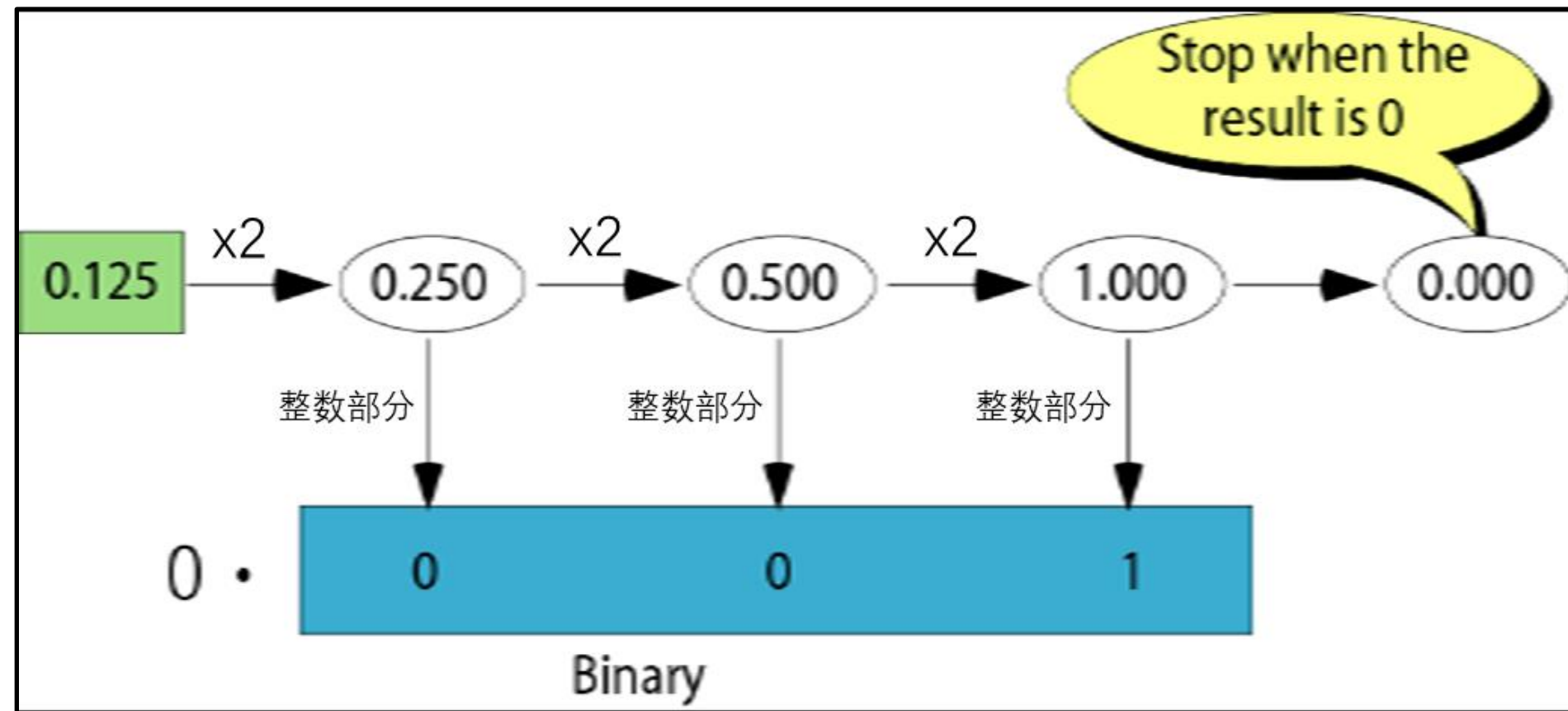
□ 单精度数 (float)

- 4个字节 (32位)
- 有效数字 6
- 指数部分 $10^{-37} \sim 10^{38}$

详细参考：
[浮点常量 - cppreference.com](http://cppreference.com)
[算术类型 - cppreference.com](http://cppreference.com)

小数 (Fraction Part) 转二进制

□ 基本原理与方法:



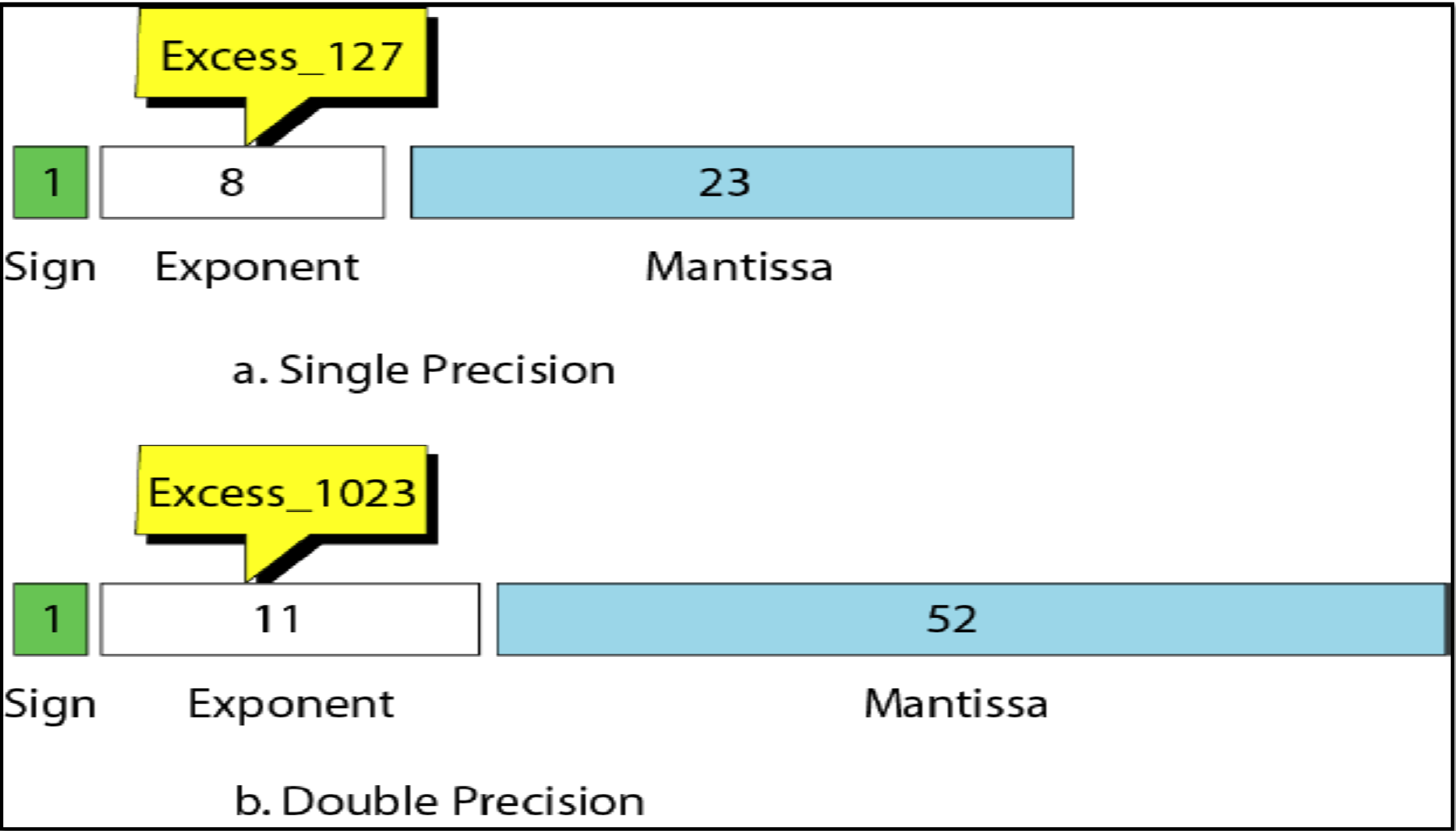
0.15
x2 0.30 -- 0
x2 0.60 -- 0
x2 1.20 -- 1 *
x2 0.40 -- 0 *
x2 0.80 -- 0 *
x2 1.60 -- 1 *
x2 1.20 -- 1 ...

$0.15 = (0.00100110011001...)_{2}$

□ 只要用二进制表示小数部分，绝大多数十进制数转二进制必然产生舍入误差（精度误差）

IEEE 745-2008 标准与浮点数内存格式

单精度数与双精度数在内存中的存储格式:



实数转为二进制表示:

- 1. 把实数的整数和小数部分分别转为二进制;
- 2. 表示成二进制科学计数法;
- 3. 按IEEE规范, 计算符号位, 指数部分, 小数部分。

例如 : 0.15
= 1.0011 0011 0011 0011 0011
001[1] * 2⁻³
注意 : 四舍五入

float x = 0.15f;

按IEEE 745规范 :

Sign 0
Exp -3 + 127 即 0x7C = 0111 1100
Manti 0011,0011,0011,0011,0011,010

结果 :
0011 1110 0001 1001
1001 1001 1001 1010
x 的二进制存储是 0x3E19999A

溢出 (Overflow) 和精度误差 (Accuracy Error)

```
/*floating accuracy*/
#include <stdio.h>

int main(void) {
    double x = 2.0e+307;
    x = x * 10;
    // 超出 double 范围的常量。
    printf("+2.0e+308 --> %e\n", x);

    float y = 123456789.0f;
    printf("123456789f --> %.8e\n", y);
    y = y + 1;
    printf("123456790f --> %.8e\n", y);
    y = y + 1;
    //此处省去100句 y = y + 1;
    y = y + 1;
    printf("1234568XXf --> %.8e\n", y);
}
```

思考：什么时候 y 的输出会改变？



中山大學
SUN YAT-SEN UNIVERSITY

补充3：变量在计算机内部

内存 (Memory) 及其组织

- 内存是计算机存储信息的电路
- 位 (bit)
 - 存储1位二进制数
- 字节 (Byte)
 - 最基本的存储单元
 - 包含 8 个 bit
- 地址 (Address)
 - 存储单元的编码
- 地址空间 (Address Space)
 - $0 \dots 2^n$ (n 为地址线位宽)
 - 例如: $n=32$ (4G)

Address	Contents
00000000	11100011
00000001	10101001
:	:
.	.
11111100	00000000
11111101	11111111
11111110	10101010
11111111	00110011

内存容量

- 内存是地址用二进制表示
- 每个内存存储单元使用一个地址
- 内存容量的单位

<i>Unit</i>	<i>Number of bytes</i>	<i>Approximation</i>
Kilobyte (K)	2^{10} bytes	10^3 bytes
Megabyte (M)	2^{20} bytes	10^6 bytes
Gigabyte (G)	2^{30} bytes	10^9 bytes
Terabyte (T)	2^{40} bytes	10^{12} bytes

- 例如：8G内存，表示有 2^{33} 个地址/存储单元/字节

变量类型、地址、值

□ 每个变量都有类型、地址、值等属性

```
/*vars-address-value*/
#include<stdio.h>

int main(void) {
    int i = 5;
    float x = 5.0f;

    printf("Addr:%p \t bin value:%#010x\n",&i,i);
    printf("Addr:%p \t bin value:%#010x\n",&x,*(int *)&x);

    return 0;
}
```

地址*	值
62FE18	00
19	00
1A	A0
1B	40
62FE1C	05
1D	00
1E	00
1F	00

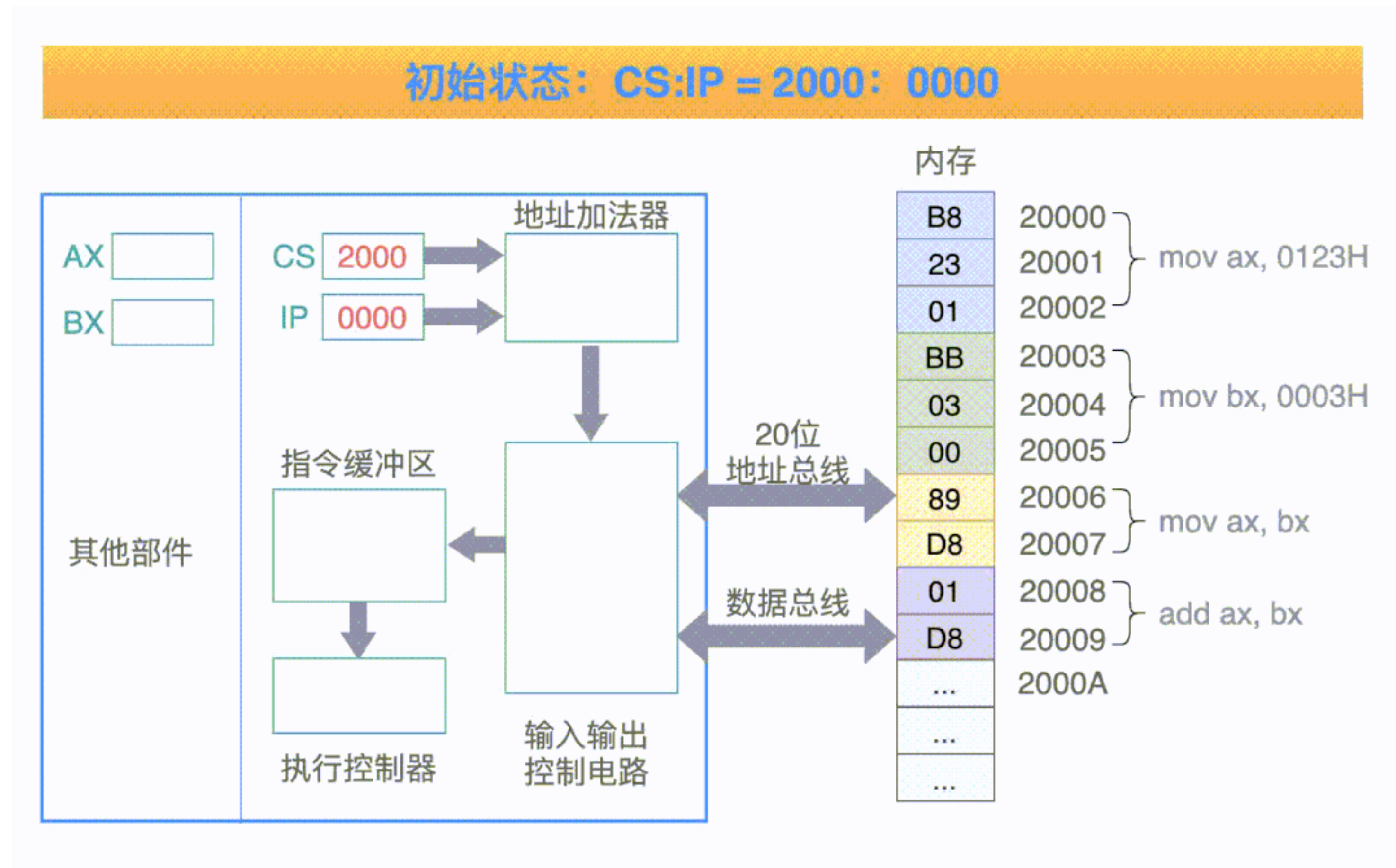
*地址随运行环境浮动

思考：把整数和浮点变量的值改为负数，其二进制内容是？

内存分区示意图



补充：CPU指令周期



谢谢大家！

邮箱: yuesh5@mail.sysu.edu.cn